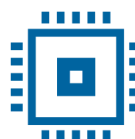




Universitatea
Transilvania
din Brașov



Universitatea
Transilvania
din Brașov

FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

Departamentul Inginerie Electrică și Știința Calculatoarelor,
Programul de studii: Electronică Aplicată

Simion Bianca - Georgiana
Vadana Alina - Antoaneta
Zaharia Delia - Ana

VERIFICARE NUMĂRĂTOR PROGRAMABIL

Braşov, 2026

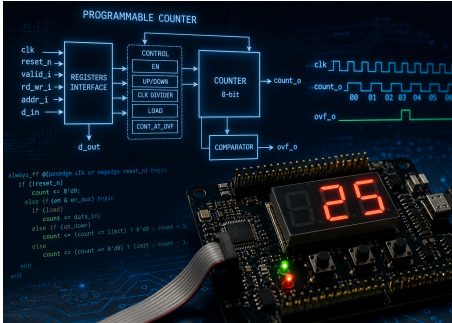
Departamentul Inginerie Electrică și Știința Calculatoarelor
Programul de studii: Electronica Aplicata

Simion Bianca-Georgiana
Vadana Alina Antoaneta
Zaharia Delia Ana

Numarator programabil

Brașov, 2026

FIȘA PROIECTULUI DE VERIFICARE

Universitatea Transilvania din Brașov Facultatea de Inginerie Electrică și Știința Calculatoarelor	Anul universitar: 2025 - 2026
Membrii: Simion Bianca Georgiana - inginer de ieșire Vadana Alina Antoaneta - arhitect Zaharia Delia Ana - inginer de intrare Coordonatori: Oana Ursu/ Alexandru Dinu	 Sursa imaginii: ChatGPT
Titlul proiectului: <i>.Numarator programabil</i>	
Sumarul funcționalităților DUT-ului: - Numărător programabil pe 8 biți. - Configurare prin registre interne. - Operații de citire și scriere prin interfața valid_i, rd_wr, addr_i, d_in și d_out. - Numărare crescătoare sau descrescătoare. - Activare/dezactivare prin bitul EN. - Încărcarea unei valori inițiale prin bitul LOAD. - Setarea limitei de numărare prin registrul limit. - Corectarea valorii încărcate dacă data_in este mai mare decât limit. - Reluarea numărării după limită prin continue_at_ovf. - Generarea semnalului ovf_o la overflow sau underflow. - Divizarea ceasului pentru numărare la 1, 2, 4 sau 8 tacte. - Citirea valorii curente prin registrul counter_value.	
Sumarul scenariilor de verificare: - Accesarea registrelor DUT-ului prin citiri și scrieri. - Verificarea tuturor adreselor disponibile. - Configurarea registrului de control. - Configurarea limitei și a valorii de încărcare. - Testarea bitului LOAD. - Verificarea numărării crescătoare. - Verificarea numărării descrescătoare. - Verificarea overflow-ului. - Verificarea reluării numărării după atingerea limitei. - Testarea cazului data_in mai mare decât limit. - Generarea de tranzații randomizate. - Acoperirea valorilor de date importante. - Monitorizarea ieșirilor count_o și ovf_o. - Colectarea coverage-ului pentru intrări și ieșiri.	

Istoricul versiunilor:

Ver-siun e	Data finalizării	Numele editorului	Motiv
0.0	20.03.2021	Alexandru Dinu	Prima versiune de model
0.1	28.03.2022	Alexandru Dinu	Template-ul a fost actualizat
1.0			Versiunea inițială
1.1			[Exemplu] s-a modificat marimea semnalului ready, de la 2 biți la 1 bit
1.2			[Exemplu] s-a adăugat o nouă stare în automatul de stări care modelează funcționarea DUT-ului în modul “automat”
2.0			O modificare majoră (se spune care) a avut loc

Cuprins

•	Locația proiectului	5
1	Proiectul digital al circuitului	6
1.1	Diagrama bloc	6
1.2	Descrierea funcționalității generale a DUT-ului	6
1.3	Interfețele DUT-ului	7
1.4	Lista cu funcționalitățile DUT-ului	8
1.5	Schema de principiu	8
1.6	Forme de undă	10
1.7	Tabel cu regiștri	10
2	Verificarea proiectului digital al circuitului	11
2.1	Arhitectura mediului de verificare	11
2.1.1	Driver pentru interfata X	12
2.1.2	Driver APB	12
2.1.3	Scoreboard / modul referinta	12
2.1.4	Monitor APB	12
2.1.5	Monitor pentru interfata X	12
2.1.6	Tranzactie pentru interfata APB	12
2.1.7	Tranzactie pentru interfața X	12
2.2	Lista cu elementele de verificat	12
2.3	Elementele de acoperire funcțională	13

2.4	Testele	13
2.5	Rezultatele obținute	14

● LOCAȚIA PROIECTULUI

Codul sursă Verilog/SystemVerilog pentru proiectul „Numărător programabil” este disponibil pe pagina de GitHub a proiectului:

https://github.com/alinavadana/Numarator_Programabil

Documentația proiectului este disponibilă pe Google Drive la următorul link:
https://docs.google.com/document/d/1M2gPhHOFu__izuwpSVvYjY0woqJcwUKT/edit

Pentru acest proiect nu a fost utilizat un mediu Edaplayground separat. Dezvoltarea și verificarea au fost realizate pe baza codului disponibil în repository-ul GitHub menționat mai sus.

1 VERIFICAREA PROIECTULUI DIGITAL AL CIRCUITULUI

Arhitectura mediului de verificare

Lista cu elementele de verificat

Elementele de acoperire funcțională

Scenarii de verificare și teste

Rezultate obținute

1.1 ARHITECTURA MEDIULUI DE VERIFICARE

Se realizează o imagine de ansamblu cu mediul de verificare.

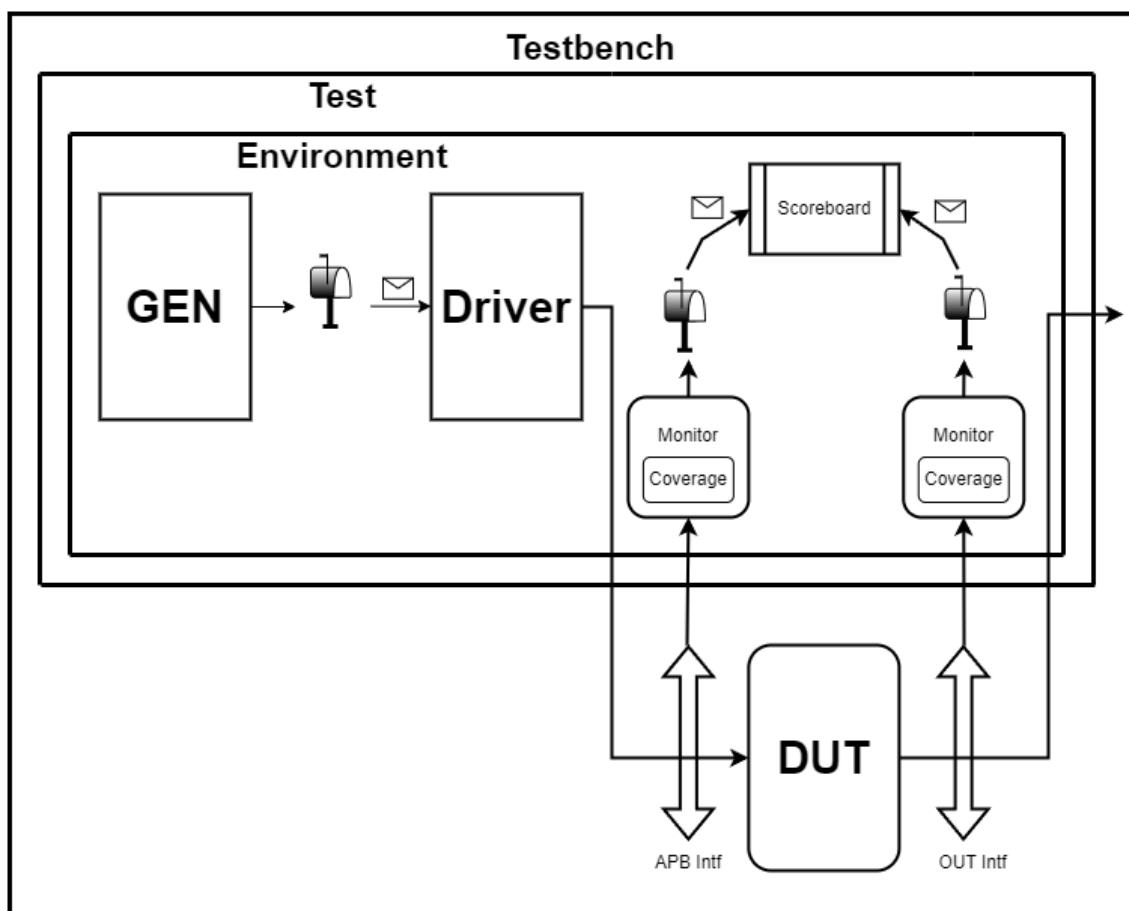


Figura 3. Imagine de ansamblu a mediului de verificare

1.1.1. Clasa Environment

Clasa environment reprezintă componenta principală a mediului de verificare. Aceasta are rolul de a crea, conecta și controla componentele folosite pentru verificarea DUT-ului.

În cadrul acestei clase sunt instanțiate componentele principale ale mediului: generatorul, driver-ul, monitorul de intrare, monitorul de ieșire și clasele de coverage. Environment-ul realizează legăturile dintre aceste componente și interfețele virtuale, apoi controlează ordinea de execuție a simulării.

La începutul simulării, environment-ul creează obiectele necesare și apelează procedura de resetare prin driver. După reset, sunt lansate în paralel procesele de generare a tranzacțiilor, aplicare a stimulilor pe interfața DUT-ului, monitorizare a semnalelor și colectare a coverage-ului funcțional.

Execuția este supravegheată printr-un mecanism de tip watchdog, care oprește simularea dacă aceasta depășește un timp maxim prestabilit. La final, environment-ul oprește procesele active și afișează raportul de coverage funcțional.



Figura 4. Schema bloc a funcționalității clasei Environment

1.1.2 Generatorul de stimuli

Clasa generator are rolul de a crea tranzacțiile care vor fi aplicate DUT-ului prin intermediul driver-ului. Aceasta generează obiecte de tip tranzacție, care conțin informațiile necesare pentru accesarea interfeței de registri a numărătorului programabil: tipul operației, adresa registrului, valoarea de date și eventuale întârzieri între tranzacții.

Generatorul poate produce atât stimuli randomizați, cât și stimuli direcționați. În cazul tranzacțiilor randomizate, valorile pentru semnalele `rd_wr_i`, `addr_i` și `d_in` sunt generate automat, respectând constrângerile impuse în clasa tranzacției. În cazul testelor direcționate, generatorul poate crea tranzacții specifice, de exemplu scrierea în registrul de control, configurarea limitei, scrierea valorii `data_in` sau citirea valorii curente a numărătorului.

După crearea unei tranzacții, generatorul o transmite către driver printr-un mailbox. Driver-ul preia tranzacția și o transformă în activitate concretă pe semnalele interfeței DUT-ului: `valid_i`, `rd_wr_i`, `addr_i` și `d_in`.

Prin folosirea generatorului se pot acoperi mai multe scenarii de verificare: scrieri și citiri în registre, accesarea tuturor adreselor, configurarea direcției de numărare, activarea sau dezactivarea numărătorului, modificarea limitei și testarea cazurilor de overflow sau underflow.

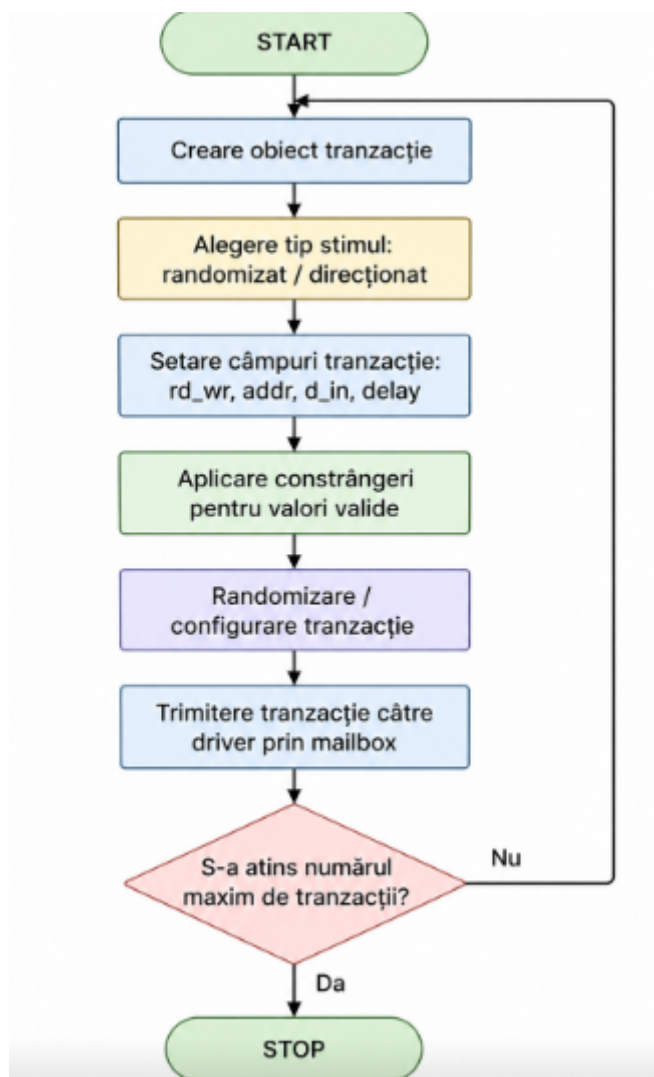


Figura 5. Schema bloc a clasei Generator

1.1.3. Driver-ul

Driver-ul este componenta care preia tranzacțiile generate de generator și le aplică efectiv pe interfața de intrare a DUT-ului. Acesta primește obiecte de tip `transaction_in` prin intermediul unui mailbox comun cu generatorul.

În timpul resetării, driver-ul aduce semnalele de intrare într-o stare cunoscută, setând `valid_i`, `rd_wr`, `addr_i` și `d_in` pe valori inactive. După terminarea resetării, driver-ul așteaptă tranzacții de la generator. Pentru fiecare tranzacție primită, acesta respectă întârzierea specificată în câmpul `delay`, apoi activează semnalul `valid_i` și aplică pe interfață adresa, tipul operației și datele de scriere, dacă tranzacția este de tip `write`.

Pentru operațiile de scriere, driver-ul pune valoarea tranzacției pe semnalul `d_in`. Pentru operațiile de citire, driver-ul setează `rd_wr` pe 1, iar valoarea citită va fi observată ulterior de monitor. După un tact de ceas, semnalul `valid_i` este dezactivat, iar driver-ul incrementează numărul de tranzacții aplicate.

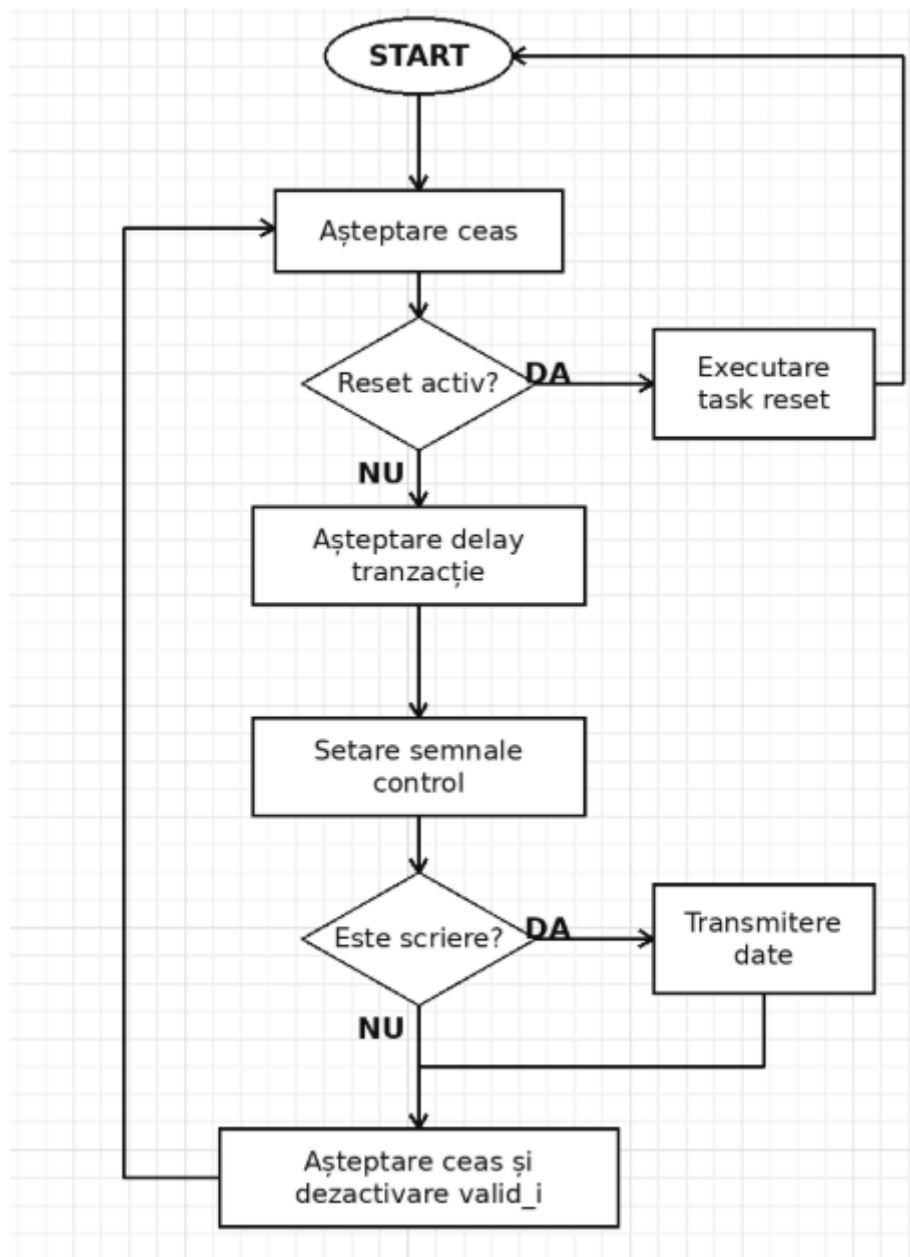


Figura 6. Schema bloc a clasei Driver

1.1.4. Monitorul de intrare

Monitorul de intrare este componenta care observă semnalele aplicate pe interfața de intrare a DUT-ului. Acesta nu modifică semnalele, ci doar le citește și reconstruiește tranzacțiile observate.

Monitorul urmărește semnalul valid_i. Atunci când valid_i este activ, monitorul creează un obiect de tip transaction_in și salvează valorile semnalelor rd_wr, addr_i și, în cazul unei operații de scriere, d_in. Dacă operația este de citire, monitorul mai așteaptă un tact de ceas pentru a captura valoarea d_out, deoarece citirea este sincronă.

După capturarea datelor, monitorul incrementează numărul de tranzacții observate, afișează informațiile pentru debug și trimite tranzacția mai departe prin mailbox. În plus, monitorul apelează clasa de coverage de intrare pentru eșantionarea tranzacției observate.

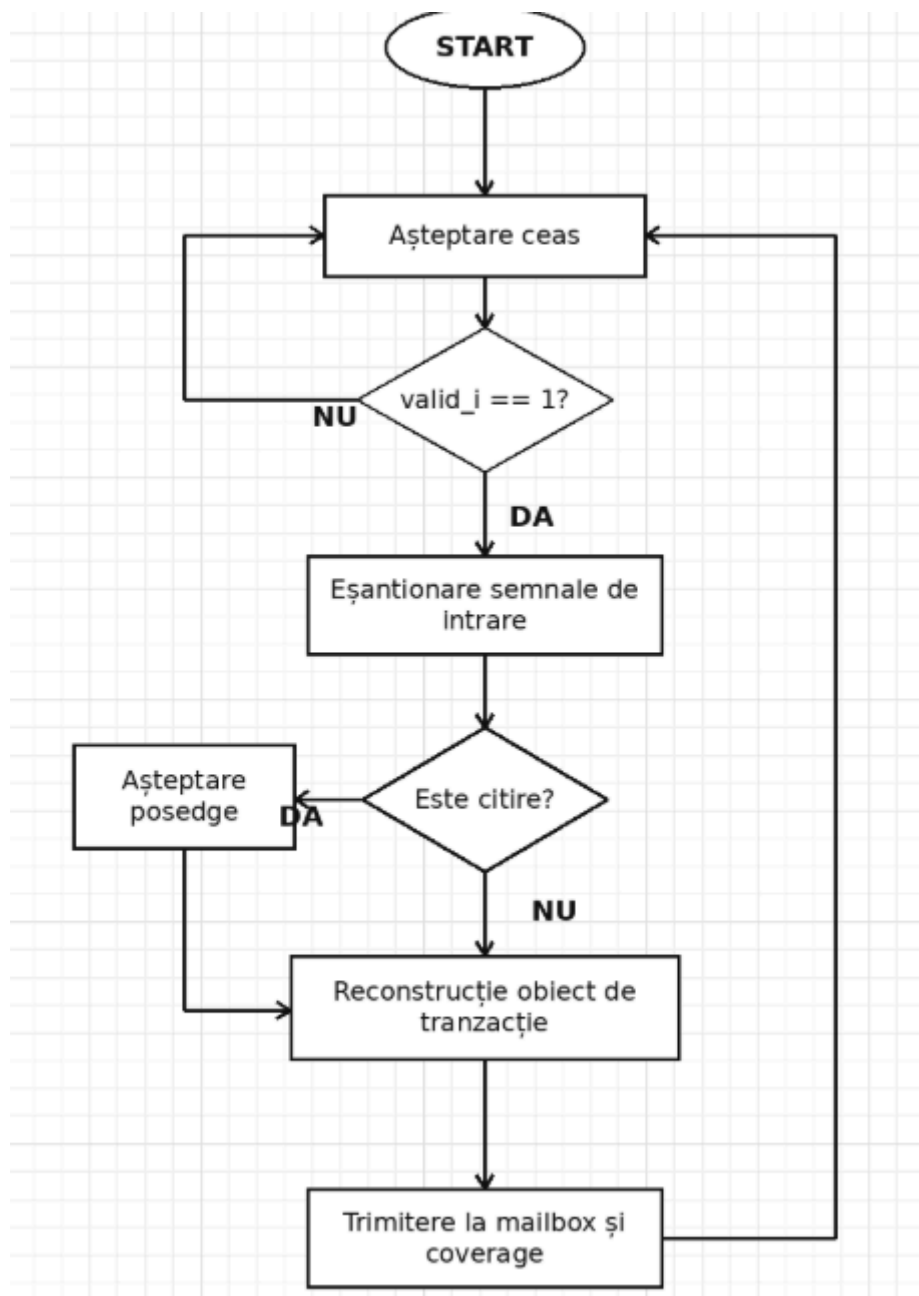


Figura 7. Schema bloc a clasei Monitor de intrare

1.1.5 Monitorul de ieșire

Monitorul de ieșire este componenta care observă semnalele generate de DUT pe interfața de ieșire. Acesta urmărește valorile semnalelor count_o și ovf_o și le salvează într-un obiect de tip transaction_out.

Monitorul de ieșire funcționează sincron cu clocking block-ul interfeței de ieșire. La fiecare eșantionare, acesta creează o tranzacție nouă, copiază în ea valoarea curentă a numărătorului și valoarea semnalului de overflow, apoi incrementează contorul de tranzacții observate.

Tranzacția observată este trimisă prin mailbox și este folosită pentru colectarea coverage-ului de ieșire. Astfel, se poate urmări dacă în simulare au fost acoperite valori importante ale numărătorului și dacă semnalul de overflow a fost activat.

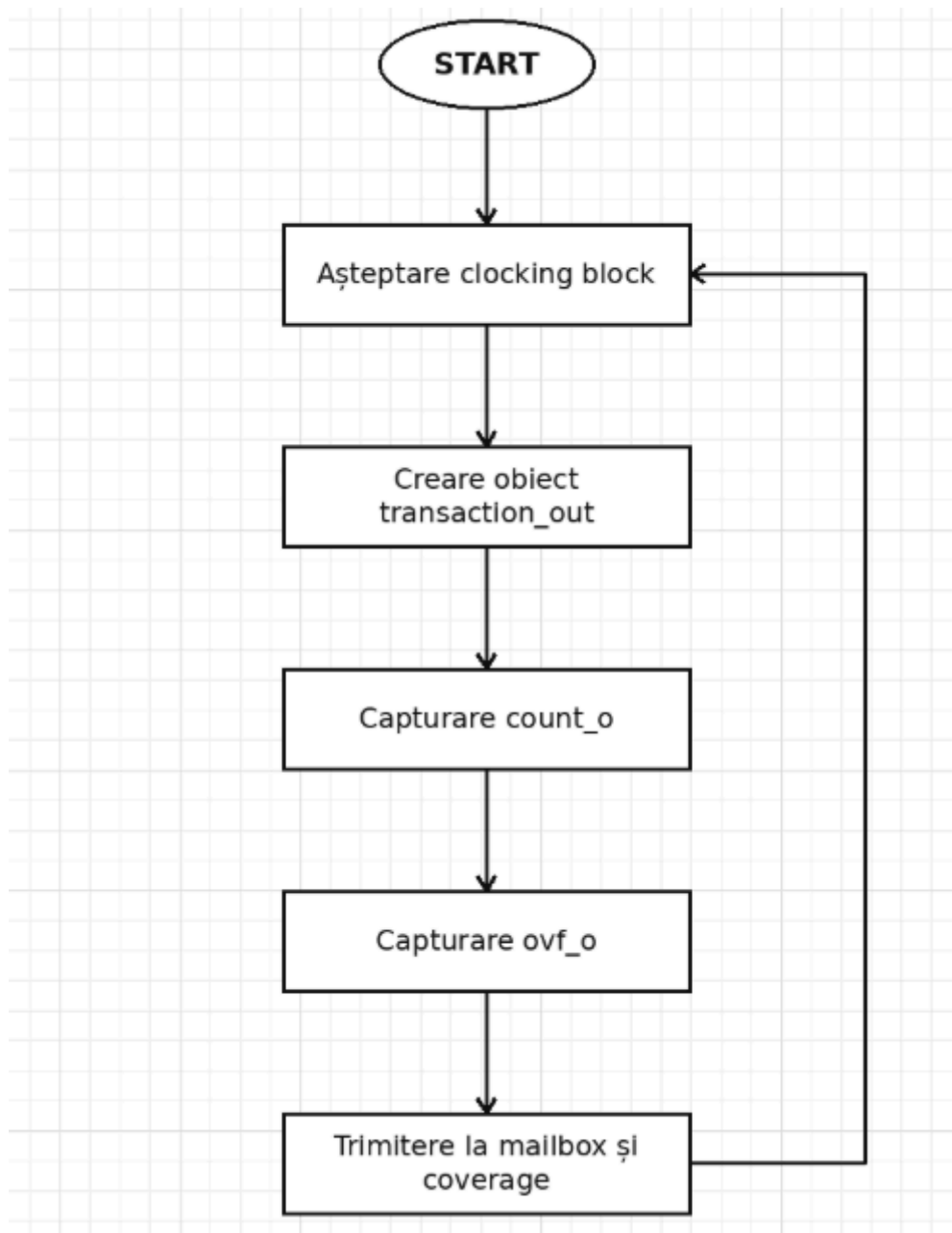


Figura 8. Schema bloc a clasei Monitor de ieșire

1.1.6 Clasa coverage_in

Clasa `coverage_in` este responsabilă cu măsurarea acoperirii funcționale pentru tranzacțiile aplicate pe interfața de intrare. Aceasta primește tranzacții de tip `transaction_in` și eșantionează câmpurile importante ale acestora.

Coverage-ul de intrare urmărește tipul operației, adresa accesată, datele scrise, datele citite și întârzierea dintre tranzacții. Pentru tipul operației sunt definite valori pentru scriere și citire. Pentru adresă sunt urmărite toate cele patru adrese ale registrelor DUT-ului. Pentru date sunt definite intervale de valori mici, medii, mari și valori extreme, cum ar fi 0 și 255.

Clasa conține și un cross între tipul operației și adresă, pentru a verifica dacă au fost realizate atât citiri, cât și scrieri pe adresele importante ale interfeței. La finalul simulării, clasa afișează procente de coverage obținute.

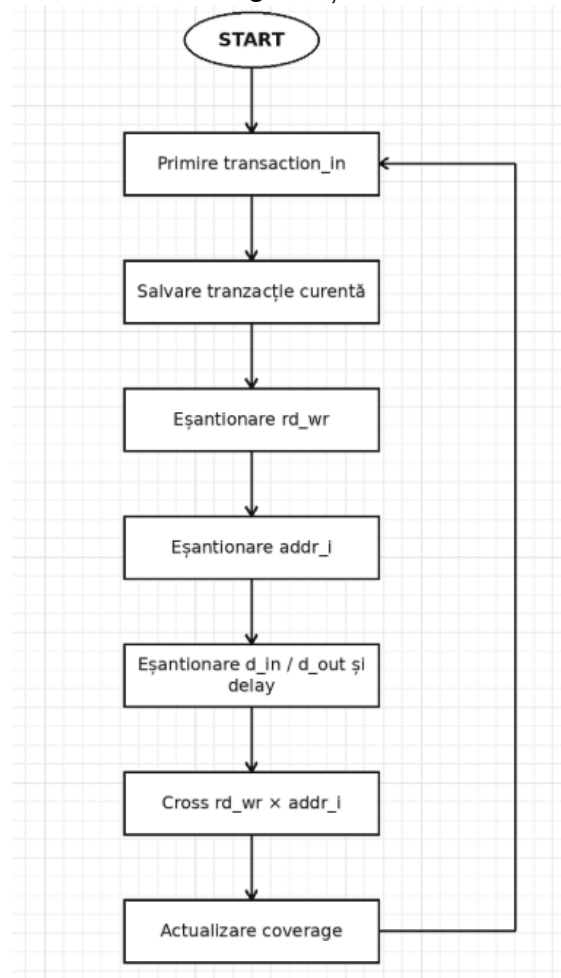


Figura 9. Schema bloc a clasei Coverage de intrare

1.1.7 Clasa coverage_out

Clasa coverage_out este responsabilă cu măsurarea acoperirii funcționale pentru semnalele de ieșire ale DUT-ului. Aceasta primește tranzacții de tip transaction_out de la monitorul de ieșire și eșantionează valorile semnalelor count_o și ovf_o.

Pentru count_o sunt urmărite valoarea minimă, valoarea maximă și mai multe intervale de valori intermediare. Acest lucru permite verificarea faptului că numărătorul nu rămâne doar într-o zonă restrânsă de valori în timpul simulării.

Pentru ovf_o sunt urmărite ambele valori posibile, 0 și 1. Astfel, se poate observa dacă în timpul simulării a fost atins și cazul în care semnalul de overflow este activat. La final, clasa afișează procentul de coverage pentru count_o, ovf_o și coverage-ul total de ieșire.

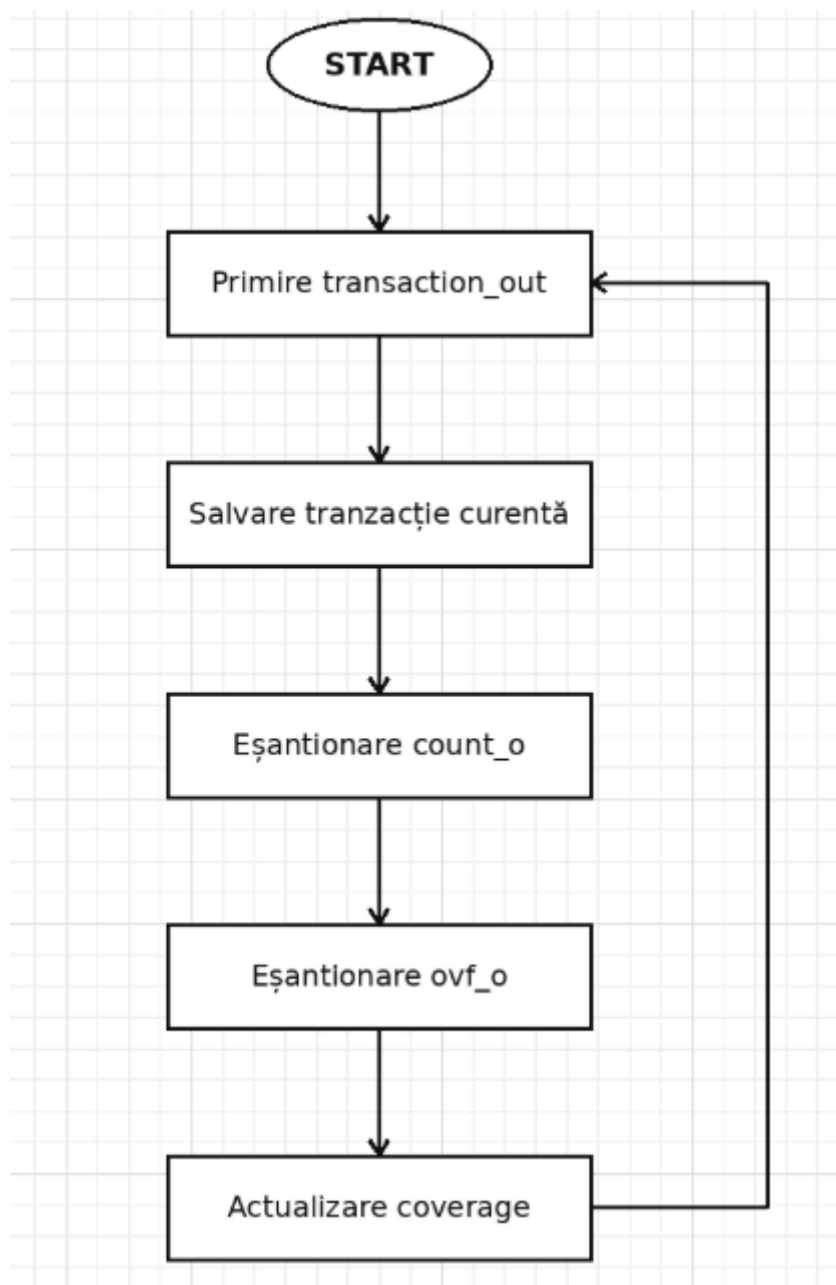


Figura 10. Schema bloc a clasei Coverage de ieșire

1.1.8. Clasa transaction_in

Clasa transaction_in definește formatul unei tranzacții aplicate pe interfața de intrare a DUT-ului. Aceasta conține câmpurile necesare pentru configurarea registrelor numărătorului: valid_i, rd_wr, addr_i, d_in, d_out și delay.

Câmpurile valid_i, rd_wr, addr_i, d_in și delay sunt declarate randomizabile, astfel încât generatorul poate crea automat combinații diferite de tranzacții. Câmpul d_out nu este randomizat, deoarece reprezintă o valoare citită de la DUT.

Clasa conține o constrângere pentru delay, astfel încât întârzierile dintre tranzacții să fie mai mari decât 0 și mai mici decât 10 tacte de ceas. De asemenea, clasa conține metode pentru afișarea tranzacției și pentru copierea acesteia într-un alt obiect.

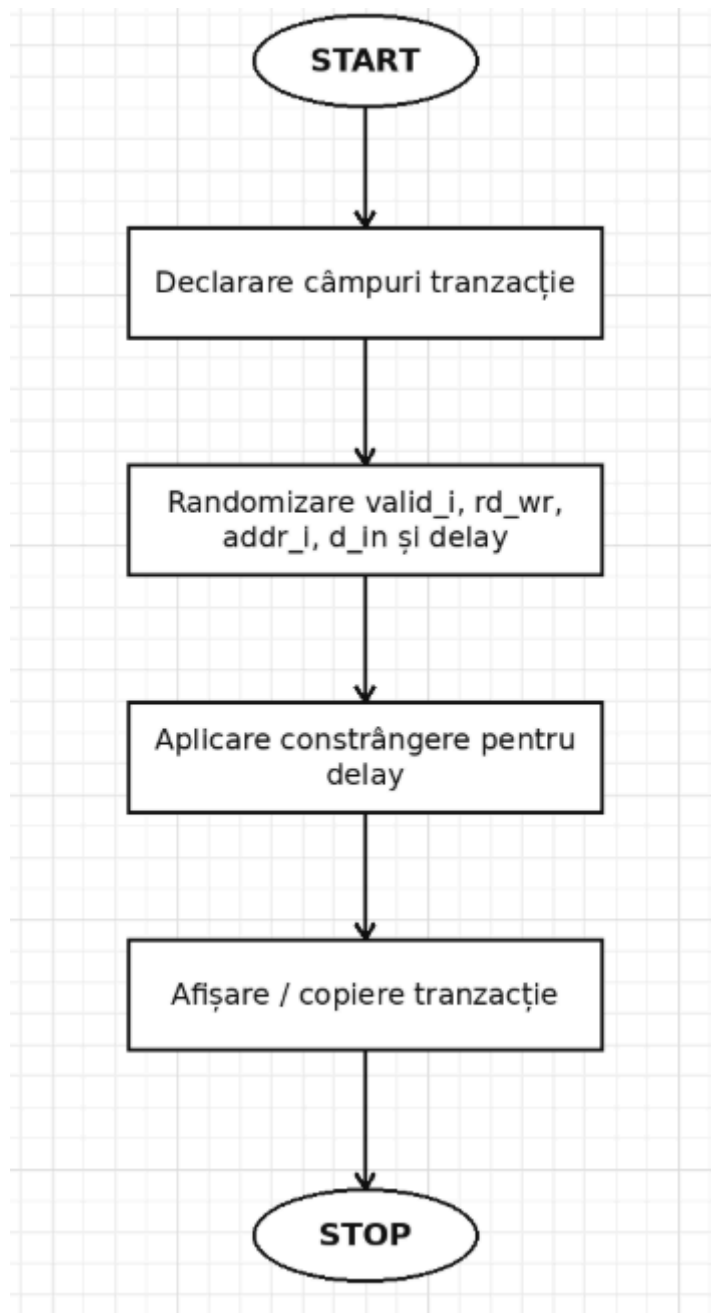


Figura 11. Schema bloc a clasei Transaction In

1.1.9. Clasa transaction_out

Clasa transaction_out definește formatul unei tranzacții observate pe interfața de ieșire a DUT-ului. Aceasta conține câmpurile count_o și ovf_o, care corespund valorii curente a numărătorului și semnalului de overflow.

Spre deosebire de transaction_in, această clasă nu este folosită pentru generarea de stimuli, ci pentru memorarea valorilor observate la ieșirea DUT-ului. Monitorul de ieșire creează obiecte de acest tip și salvează în ele valorile citite de pe interfață.

Clasa conține și o funcție de copiere, folosită pentru a crea o copie separată a tranzacției observate. De asemenea, conține o metodă de afișare după randomizare, deși în fluxul actual valorile sunt capturate de la DUT, nu generate random.

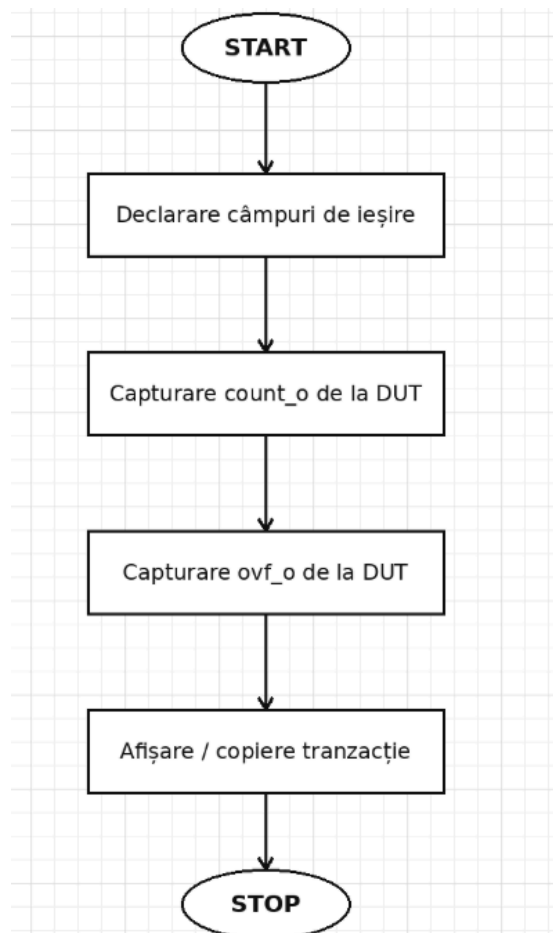


Figura 12. Schema bloc a clasei Transaction Out

1.1.10. Tranzacție pentru Interfața de intrare

Această componentă definește pachetul de date sub formă de clasă, numit `transaction_in`, care încapsulează stimulii și comenzile aplicate către DUT. Obiectul de tranzacție conține semnalele necesare pentru accesarea interfeței de regiștri a numărătorului programabil.

Clasa `transaction_in` este folosită de generator pentru crearea tranzacțiilor, de driver pentru aplicarea acestora pe interfața DUT-ului și de monitorul de intrare pentru reconstruirea tranzacțiilor observate.

Tranzacția conține informații despre validitatea transferului, tipul operației, adresa registrului accesat, datele de intrare, datele citite și întârzierea dintre tranzacții. O parte dintre câmpuri sunt randomizabile, pentru a permite generarea automată a stimulilor.

Clasa conține și o constrângere pentru delay, astfel încât întârzierea dintre tranzacții să fie mai mare decât 0 și mai mică decât 10 tacte de ceas.

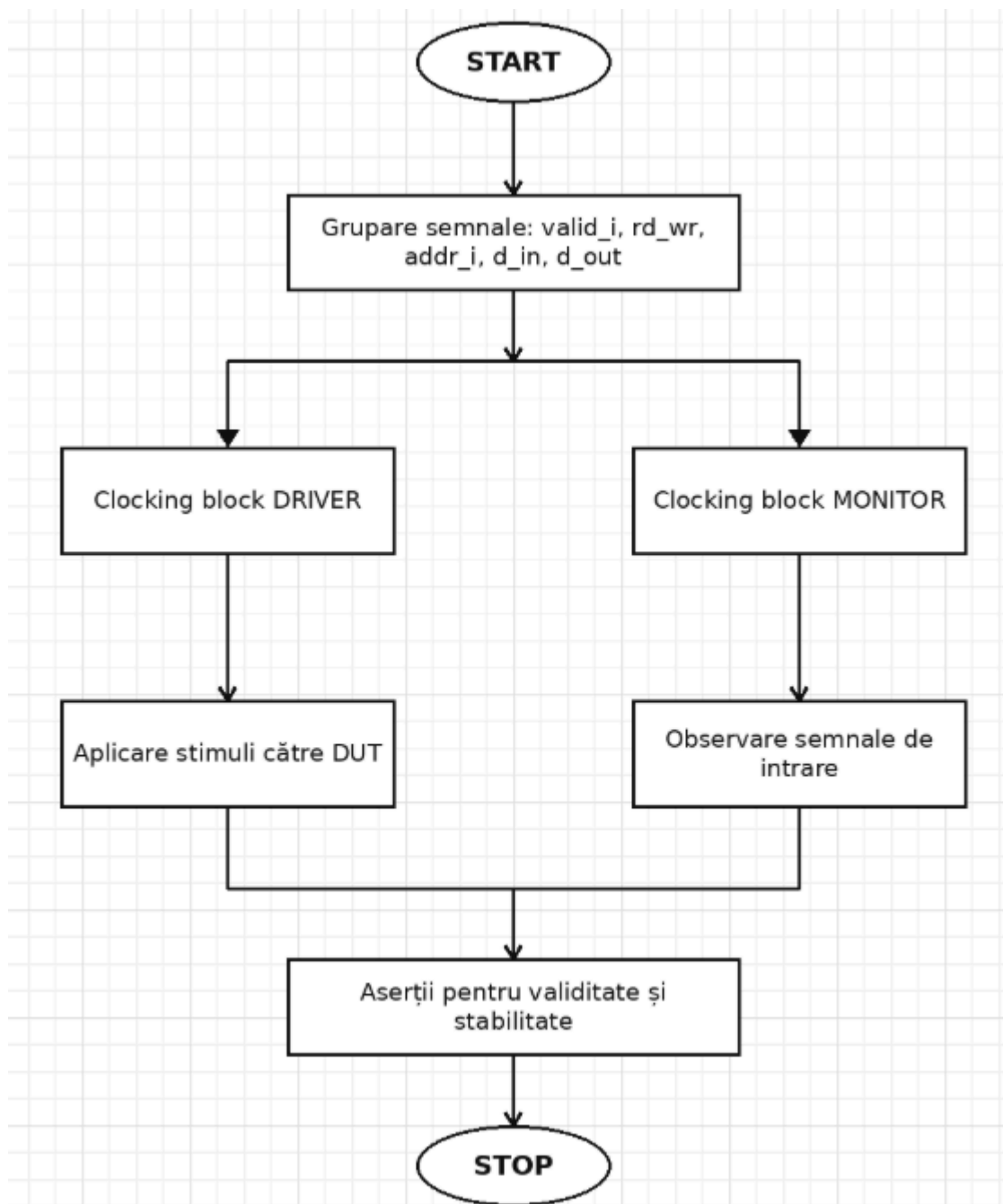


Figura 13. Schema bloc a interfeței de intrare

Nume element	Tip	Descriere
valid_i	Variabilă(rand bit)	Indică validitatea tranzacției pe interfața de intrare. Când este 1, DUT-ul ia în considerare operația curentă.

rd_wr	Variabilă(rand bit)	Definește tipul operației: 0 pentru scriere și 1 pentru citire.
addr_i	Variabilă(rand bit [1:0])	Adresa registrului vizat (00: CTL, 01: COUNTER, 10: LIMIT, 11: DATA_IN).
d_in	Variabilă(rand bit [7:0])	Datele transmise către DUT în cazul unei operații de scriere.
d_out	Variabilă(bit [7:0])	Datele citite de la DUT în urma unei operații de citire. Nu este randomizată.
cnt	Variabilă static int	Contor static folosit pentru numerotarea tranzacțiilor afișate în simulare.
delay	Variabilă(rand bit)	Numărul de tacte de ceas așteptate între două tranzacții.
delay_constr	Constrângere	Limitează valoarea lui delay astfel încât $\text{delay} > 0 \ \&\& \ \text{delay} < 10$
display()	Funcție	Afișează în consolă valorile câmpurilor tranzacției.
post_randomize()	Funcție	Este apelată după randomizare și apelează funcția display ()
do_copy()	Funcție	Creează și returnează o copie a obiectului de tip transaction_in

1.1.11. Tranzacție pentru Interfața de ieșire

Interfața de ieșire grupează semnalele produse de DUT: count_o și ovf_o. Aceasta este folosită de monitorul de ieșire pentru a observa comportamentul numărătorului în timpul simulării.

Interfața conține un clocking block pentru monitor, prin care valorile de ieșire sunt citite sincron cu ceasul. De asemenea, sunt definite aserții SystemVerilog pentru verificarea unor relații importante între count_o și ovf_o.

Aserțiile urmăresc activarea semnalului de overflow în situațiile în care numărătorul trece între limitele extreme. Astfel, interfața de ieșire nu are doar rol de conectare, ci și rol de verificare a comportamentului semnalelor observabile.

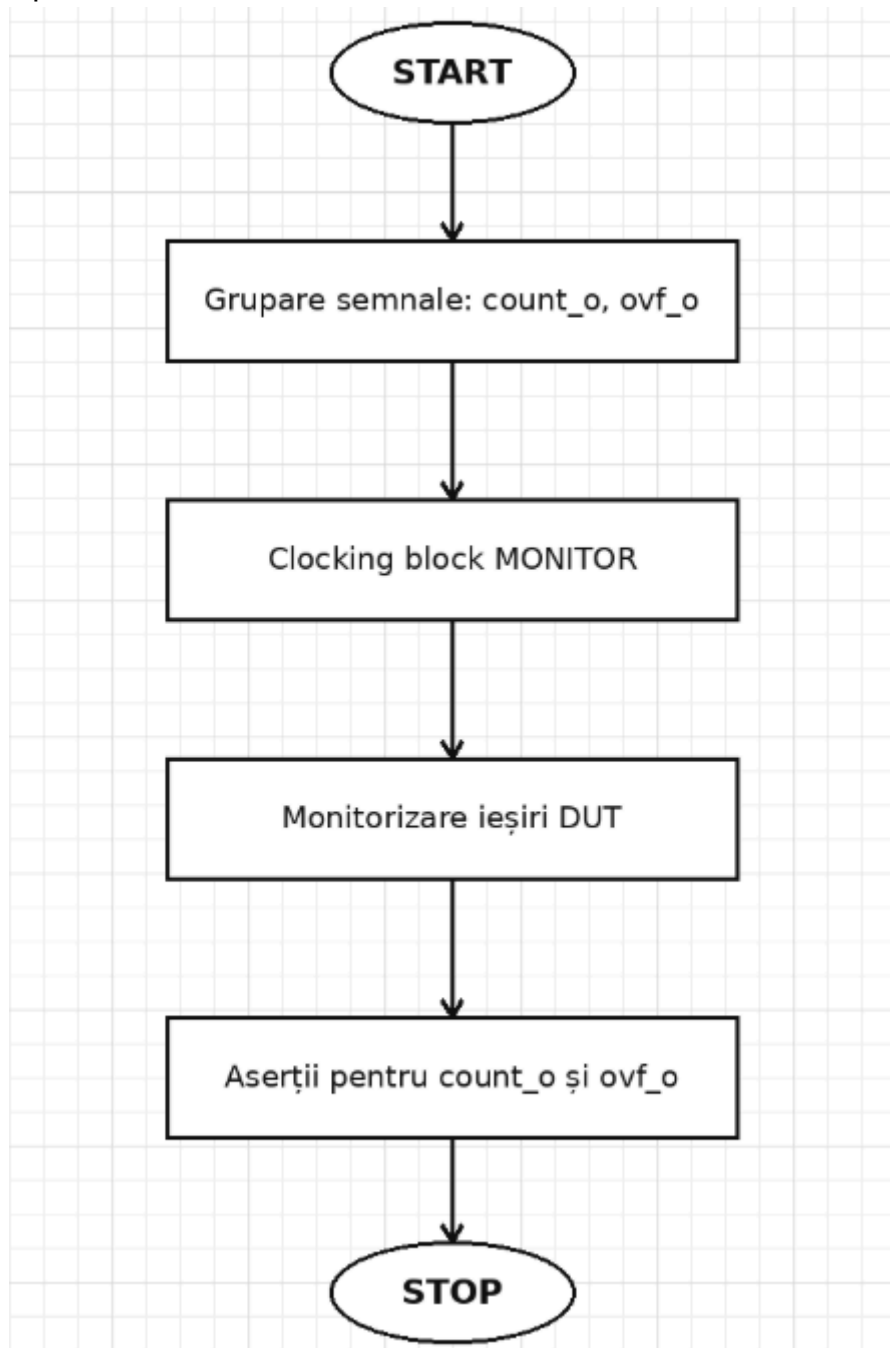


Figura 14. Schema bloc a interfeței de ieșire

Nume element	Tip	Descriere
count_o	Variabilă (bit [7:0])	Valoarea curentă a numărătorului, capturată de la ieșirea DUT-ului.
ovf_o	Variabilă (bit)	Semnalul de overflow/underflow generat de DUT.
cnt	Variabilă (int)	Contor static folosit pentru numerotarea tranzacțiilor afișate în simulare.
delay	Variabilă (int)	Câmp folosit pentru memorarea unei întârzieri asociate tranzacției de ieșire.
post_randomize()	Funcție	Afișează valorile câmpurilor tranzacției. În fluxul actual, valorile sunt capturate de la DUT, nu generate random.
do_copy()	Funcție	Creează și returnează o copie a obiectului de tip transaction_out

1.1.12. Testbench-ul

Testbench-ul este componenta de nivel superior care conectează DUT-ul cu mediul de verificare. Acesta generează semnalul de ceas, generează resetul, instanțiază interfețele și conectează DUT-ul la aceste interfețe.

În testbench este instanțiat modulul counter_programabil, iar semnalele acestuia sunt conectate la interfața de intrare și la interfața de ieșire. Interfața de intrare transportă semnalele de configurare ale DUT-ului, iar interfața de ieșire transportă semnalele count_o și ovf_o.

Tot în testbench este instanțiat programul de test, care pornește mediul de verificare. La final, testbench-ul generează fișierul dump.vcd, folosit pentru vizualizarea formelor de undă.

1.1.13 Programele de test

Programele de test definesc scenariile care vor fi rulate în simulare. Acestea instanțiază mediul de verificare, configurează generatorul și pornesc execuția prin apelarea metodei run din environment.

Testul randomizat setează un număr de tranzacții care trebuie generate automat și apoi pornește mediul. Testul simplu realizează un scenariu direcționat, în care se scrie o valoare în registrul de control, apoi se citește același registru. Testul de limită configurează registrul limit, setează o valoare de încărcare și activează numărătorul prin registrul de control.

Prin aceste teste se pot verifica atât funcționarea generală a interfeței de registre, cât și cazuri particulare legate de configurarea limitei și încărcarea valorii în numărător.

1.2 LISTA CU ELEMENTELE DE VERIFICAT

În cadrul mediului de verificare au fost definite aserții SystemVerilog în interfețele DUT-ului. Acestea au rolul de a verifica validitatea semnalelor în timpul tranzacțiilor și corelarea semnalelor de ieșire ale numărătorului cu situațiile de overflow.

Aserțiile sunt plasate în interfața de intrare și în interfața de ieșire, deoarece aceste zone sunt punctele principale de observare și control ale DUT-ului.

Aceste aserții sunt definite în out_interface.sv, unde sunt monitorizate semnalele count_o și ovf_o.

Nume variabile	Descriere
ovf_asserted	Verifică faptul că semnalul ovf_o, atunci când este activ, apare într-un context valid de overflow/underflow. Aserția verifică dacă count_o este 0 sau dacă valoarea anterioară a lui count_o a fost 0. Astfel, se urmărește ca activarea semnalului ovf_o să fie corelată cu trecerea numărătorului printr-o valoare limită.
asertia_ovf_asserted	Este instanțierea aserției pentru proprietatea ovf_asserted. Dacă proprietatea nu este respectată, simulatorul afișează un mesaj de eroare pentru interfața de ieșire.
ovf_asserted_c	Este un cover_property asociat proprietății ovf_asserted. Acesta verifică dacă situația urmărită de aserție a fost atinsă cel puțin o dată în timpul simulării.
count_asserted	Verifică activarea semnalului ovf_o atunci când numărătorul trece între valorile extreme. Proprietatea urmărește tranzițiile 255->0 și 0 -> 255, iar în aceste cazuri ovf_o trebuie să fie activ.
asertia_count_asserted	Este instanțierea aserției pentru proprietatea count_asserted. Dacă tranziția între valorile extreme are loc fără activarea lui ovf_o, se generează eroare.

count_asserted_c	Este un cover_property pentru proprietatea count_asserted. Acesta confirmă că scenariul de trecere între valorile extreme a fost executat în simulare.
------------------	--

Tabelul 5. Aserțiile de pe interfața de ieșire

Nume variabile	Descriere
steady_addr_i_during_transaction	Verifică faptul că, în timpul unei tranzacții valide (valid_i = 1), semnalul addr_i nu are valori necunoscute X sau stări de înaltă impedanță Z.
asertia_steady_addr_i_during_transaction	Este instanțierea aserției pentru proprietatea steady_addr_i_during_transaction. Dacă addr_i devine invalid în timpul unei tranzacții, se afișează eroare.
steady_addr_i_during_transaction_C	Este un cover_property care confirmă că proprietatea pentru verificarea adresei a fost activată cel puțin o dată în simulare.
steady_d_in_during_transaction	Verifică faptul că, în timpul unei scrieri valide (valid_i = 1 și rd_wr = 0), , semnalul d_in nu are valori X sau Z.
asertia_steady_d_in_during_transaction	Este instanțierea aserției pentru proprietatea steady_d_in_during_transaction. Dacă datele de intrare sunt invalide în timpul unei scrieri, se generează eroare.
steady_d_in_during_transaction_C	Este un cover_property care verifică dacă scenariul de scriere validă a fost exercitat în simulare.
steady_d_out_during_transaction	Verifică faptul că, în timpul unei citiri valide (valid_i = 1 și rd_wr = 1), semnalul d_out nu are valori X sau Z în tactul următor. Este folosit operatorul `
asertia_steady_d_out_during_transaction	Este instanțierea aserției pentru proprietatea steady_d_out_during_transaction. Dacă d_out este invalid după o citire, se afișează eroare.
steady_d_out_during_transaction_C	Este un cover_property care confirmă că scenariul de citire a fost executat cel puțin o dată.
steady_rd_wr_i_during_transaction	Verifică faptul că, în timpul unei tranzacții valide, semnalul rd_wr nu are valori necunoscute X sau stări Z.
asertia_steady_rd_wr_i_during_transaction	Este instanțierea aserției pentru proprietatea steady_rd_wr_i_during_transaction. Dacă semnalul de control rd_wr este invalid, se generează eroare.

steady_rd_wr_i_during_transaction_C	Este un cover_property care verifică dacă proprietatea pentru rd_wr a fost activată în simulare.
write_bus_stable	Verifică stabilitatea semnalelor folosite la scriere. Atunci când există o scriere validă (valid_i = 1 și rd_wr = 0), în tactul următor semnalele addr_i, d_in și rd_wr trebuie să rămână stabile.

Tabelul 6. Aserțiile de pe interfața de intrare

1.3 ELEMENTELE DE ACOPERIRE FUNCȚIONALĂ

Elementele de acoperire funcțională au rolul de a măsura cât de bine au fost exercitate scenariile importante ale DUT-ului în timpul simulării. Pentru acest proiect, coverage-ul este împărțit în două componente principale: coverage pentru interfața de intrare și coverage pentru interfața de ieșire.

Coverage-ul de intrare urmărește tranzacțiile aplicate către DUT, precum tipul operației, adresa accesată, datele transmise și întârzierea dintre tranzacții. Coverage-ul de ieșire urmărește comportamentul observabil al numărătorului, adică valoarea count_o și semnalul ovf_o.

Elementele de coverage sunt definite în clasele coverage_in și coverage_out și sunt eșantionate pe baza tranzacțiilor observate de monitoare.

1.3.1. Coverage_in

Clasa coverage_in are rolul de a măsura acoperirea funcțională pentru tranzacțiile observate pe interfața de intrare a DUT-ului. Aceasta primește obiecte de tip transaction_in de la monitorul de intrare și eșantionează câmpurile importante ale tranzacției.

Coverage-ul de intrare urmărește tipul operației, adresa registrului accesat, datele scrise pe interfață, datele citite de la DUT și întârzierea dintre tranzacții. Astfel, se poate verifica dacă simularea a acoperit atât operații de scriere, cât și operații de citire, pe toate adresele importante ale numărătorului programabil.

Pentru câmpul d_in, adică datele scrise pe interfața de intrare, sunt de interes următoarele intervale:

0 → cea mai mică valoare;
1–126 → valoare mică;
127–190 → valoare medie;
191–254 → valoare mare;
255 → limita maximă.

Pentru câmpul delay sunt de interes următoarele intervale:

0 → fără delay;
1–3 → delay mic;
4–6 → delay mediu;
7–9 → delay mare;
10–\$ → delay foarte mare;
default → valori în afara intervalelor definite.

Cover Point	Descriere
rd_wr_cp	Monitorizează tipul operației efectuate asupra registrelor DUT-ului: scriere sau citire.
address_cp	Urmărește adresele accesate pe interfața de intrare: 0,1,2,3.
write_data_cp	Grupează valorile scrise pe d_in în intervale: valoare minimă, valori mici, medii, mari și valoare maximă.
read_data_cp	Monitorizează valorile citite pe d_out, incluzând valoarea minimă, valoarea maximă și intervale automate pentru restul valorilor.
delay_cp	Măsoară timpul de așteptare dintre tranzacții, împărțit în intervale de delay.
rd_wr_x_addr	Verifică dacă au fost realizate operații de citire și scriere pe adresele DUT-ului.

Tabelul 6. Elementele de acoperire funcțională pentru interfața de intrare

1.3.2 Coverage_out

Clasa coverage_out are rolul de a măsura acoperirea funcțională pentru semnalele observate la ieșirea DUT-ului. Aceasta primește obiecte de tip transaction_out de la monitorul de ieșire și eșantionează valorile semnalelor count_o și ovf_o.

Coverage-ul de ieșire urmărește dacă numărătorul a ajuns în valori relevante pe parcursul simulării și dacă semnalul de overflow/underflow a fost activat. Astfel, se poate verifica dacă testele au exercitat atât funcționarea normală a numărătorului, cât și cazurile de limită.

Pentru câmpul count_o sunt urmărite valoarea minimă, valoarea maximă și intervale intermediare ale domeniului de numărare. Deoarece numărătorul este pe 8 biți, valorile posibile sunt cuprinse între 0 și 255.

Pentru câmpul ovf_o sunt urmărite cele două valori posibile ale semnalului: 0 și 1. Valoarea 0 indică lipsa unui eveniment de overflow/underflow, iar valoarea 1 indică apariția unui astfel de eveniment.

Cover Point	Descriere	Interval de interes (Bins)
count_o_cp	Monitorizează valorile atinse de numărător în timpul simulării.	0, 255, 3, plus 4 intervale automate în [1 :254]
overflow _o_ cp	Monitorizează semnalul de overflow/underflow generat de DUT	0, 1

Tabelul 7. Elementele de acoperire funcțională pentru interfața de ieșire

1.4 SCENARII DE VERIFICARE ȘI TESTE

În această secțiune sunt prezentate scenariile propuse pentru verificarea DUT-ului și testele implementate în mediul de verificare. Scenariile au fost alese astfel încât să acopere funcționalitățile principale ale numărătorului programabil: resetarea, accesul la registre, citirea și scrierea prin interfață, configurarea limitei, încărcarea unei valori inițiale și observarea semnalelor de ieșire count_o și ovf_o.

Testele implementate folosesc mediul de verificare format din generator, driver, monitoare și clase de coverage. Generatorul poate produce tranzacții randomizate sau tranzacții direcționate pentru accesarea registrelor DUT-ului.

Titlul scenariului	Descrierea scenariului propus
Resetarea DUT-ului	Se activează semnalul de reset și se verifică faptul că DUT-ul pornește dintr-o stare cunoscută. Se urmărește inițializarea numărătorului și a registrelor interne conform specificației.
Accesarea registrelor prin interfață	Se verifică funcționarea interfeței de regiștri folosind semnalele valid_i, rd_wr, addr_i, d_in și d_out. Se urmărește realizarea operațiilor de citire și scriere.
Scrierea registrului de control	Se scrie o valoare în registrul de control de la adresa 0,, pentru configurarea numărătorului. Prin acest registru se pot seta enable-ul, direcția de numărare, divizorul de ceas, comportamentul la overflow și bitul de load.
Citirea registrului de control	După scrierea registrului de control, se realizează o citire de la aceeași adresă pentru a observa valoarea returnată pe d_out.
Configurarea limitei de numărare	Se scrie o valoare în registrul limit, aflat la adresa 2, pentru a modifica limita până la care poate număra DUT-ul.
Configurarea valorii de încărcare	Se scrie o valoare în registru data_in, aflat la adresa 3, pentru a seta valoarea care poate fi încărcată în numărător atunci când bitul load este activ.
Încărcarea unei valori în numărător	Se configurează registrul data_in, apoi se setează bitul load din registrul de control. Se urmărește comportamentul lui count_o după comanda de încărcare.
Tranzacții randomizate	Se generează un număr de tranzacții randomizate pentru a acoperi combinații diferite de operații, adrese, date și întârzieri între tranzacții.
Monitorizarea ieșirilor DUT-ului	Se observă semnalele count_o și ovf_o prin monitorul de ieșire și se colectează coverage pentru valorile generate de numărător.
Colectarea coverage-ului funcțional	Se verifică dacă simularea a acoperit operații de citire/scriere, adresele registrelor, valori de date, întârzieri și valori relevante ale ieșirilor.
Citirea tuturor registrelor DUT-ului	Se citesc toate registrele accesibile ale DUT-ului, folosind adresele 0, 1,2 și 3, pentru a verifica accesarea completă a hărții de registre.
Acoperirea valorilor	Se scriu și se citesc valori reprezentative pentru câmpul d_in,

de date	acoperind valoarea minimă, valori mici, valori medii, valori mari și valoarea maximă pe 8 biți.
Numărare crescătoare	Se configurează registrul de control cu EN = 1 și UP_DOWN = 1, pentru a verifica funcționarea numărătorului în sens crescător.
Numărare descrescătoare	Se configurează registrul de control cu EN = 1 și UP_DOWN = 0, pentru a verifica funcționarea numărătorului în sens descrescător.
Generarea semnalului de overflow	Se setează o limită mică și se activează bitul continue_at_ovf, astfel încât numărătorul să ajungă rapid la limită și să activeze semnalul ovf_o.
Reluarea numărării după atingerea limitei	Se configurează bitul continue_at_ovf = 1, pentru a verifica faptul că numărătorul reia numărarea după atingerea limitei configurate.
Valoare <code>data_in</code> mai mare decât <code>limit</code>	Se setează registrul limit la o valoare mică și se încearcă scrierea unei valori mai mari în registrul data_in, pentru a observa comportamentul DUT-ului în acest caz limită.

Tabelul 8. Lista scenariilor proiectate pentru verificarea DUT-ului

Numele fișierului care conține testul	Descrierea testului realizat
test.sv	Test randomizat general. Generează tranzații de citire/scriere cu adrese, date și întârzieri randomizate.
test_simplu.sv	Test direcționat pentru scrierea și citirea registrului de control de la adresa 0.
test_limita.sv	Configurează registrul limit, registrul data_in și activează încărcarea valorii prin bitul LOAD.
test_up.sv	Configurează numărătorul pentru numărare crescătoare prin EN = 1 și UP_DOWN = 1.
test_numarare_descrescator.sv	Configurează numărătorul pentru numărare descrescătoare prin EN = 1 și UP_DOWN = 0.
test_overflow_up.sv	Forțează apariția overflow-ului la numărare crescătoare, folosind o limită mică și continue_at_ovf = 1.
test_reluare_numarare.sv	Verifică reluarea numărării după atingerea limitei, cu continue_at_ovf = 1.
test_data_bins.sv	Scrie și citește valori reprezentative pentru acoperirea intervalelor de date: minim, mic, mediu, mare și maxim.
test_read_all_registers.	Citește toate registrele DUT-ului, acoperind adresele 0, 1, 2 și

sv	3.
test_data_in_mai_mare_decat_limita.sv	Verifică situația în care data_in este mai mare decât limit.

Tabelul 9. Lista testelor implementate pentru verificarea DUT-ului

1.5 REZULTATE OBȚINUTE

Rezultatele privesc acoperirea funcțională desfășurarea proiectelor per ansamblu și alte aspecte de interes.

În urma rulării testului test_complete.sv s-a obținut valoarea de 100% pentru elementul de acoperire funcțională privind valorile latenței.

interfața	element coverage	valoare maximă obținută	test
intrare	address coverage	100%	test.sv
intrare	rd/wr_coverage	100%	test_simplu.sv
intrare	write data coverage	100%	test_data_bins.sv
intrare	read data coverage	83.33%	test_data_bins.sv
intrare	delay coverage	20%	test_simplu.sv
intrare	overall coverage	78.47%	test_read_all_registers.sv
ieșire	count coverage	100%	test_read_all_registers.sv
ieșire	overflow coverage	100%	test_simplu.sv
ieșire	overall out cov	92.86%	test_simplu.sv